

From Universal Crypto Bridge to Deterministic Quantum-Resilient Cryptographic Operations

A Complete Intellectual Lineage Map of DSOS-PQC Framework

Author: Leon Calvin Long II

Affiliation: Independent Researcher / Inventor

Date: August 2026

Document Type: Intellectual Lineage Map

INTELLECTUAL LINEAGE MAP — VERSION 1.0

|

CONFIDENTIAL DRAFT

Purpose Statement:

This document establishes and formalizes the continuous intellectual lineage from Universal Crypto Bridge (UCB) through Deterministic Symmetric Operating System Science (DSOS) and its derived frameworks, culminating in Deterministic Quantum-Resilient Cryptographic Operations (DQCO) and Post-Quantum Cryptography (PQC) alignment. It serves as a citable provenance record.

Table of Contents

Abstract

Section 1 — Introduction: The Problem Space

Section 2 — Lineage Node 1: Universal Crypto Bridge (UCB)

Section 3 — Lineage Node 2: DSOS Science

Section 4 — Lineage Node 3: Deterministic Governance

Section 5 — Lineage Node 4: Neural-Mesh Substrates

Section 6 — Lineage Node 5: JasperOS and SquirrelOS

Section 7 — Lineage Node 6: DQCO

Section 8 — Chronological Lineage Timeline

Section 9 — Architectural Dependency Map

Section 10 — Conceptual Novelty Analysis

Section 11 — PQC Alignment Statement

Section 12 — Provenance Record

Section 13 — Conclusion

Appendix A — Glossary of Core Terms

Appendix B — Abbreviations Index

Abstract

This document presents a formal intellectual lineage map tracing the sequential and logically necessary evolution of a body of work in deterministic cryptographic systems science. The lineage proceeds from its foundational origin — the Universal Crypto Bridge (UCB) — through a series of conceptually dependent frameworks, culminating in Deterministic

Quantum-Resilient Cryptographic Operations (DQCO) and alignment with the National Institute of Standards and Technology (NIST) Post-Quantum Cryptography (PQC) standardization initiative.

The UCB established the primordial architectural principle of the entire lineage: that cryptographic interoperability must be deterministically provable rather than probabilistically assured. This principle — determinism as a first-class cryptographic property — was subsequently elevated by DSOS Science (Deterministic Symmetric Operating System Science) into a full governing epistemology for operating system design, wherein every security-critical system behavior is anchored to a verifiable deterministic proof tree rather than heuristic policy or probabilistic enforcement.

From DSOS Science, two parallel derivations emerged: Deterministic Governance, which formalized policy enforcement as a class of mathematical proof objects, and Neural-Mesh Substrates, which extended DSOS determinism horizontally across distributed, biologically-inspired mesh architectures. These two derivations converged in the concrete operating system implementations of JasperOS and SquirrelOS — deployable, hardware-agnostic OS instances that demonstrate DSOS Science's transition from theoretical framework to production-grade systems engineering.

The entirety of this accumulated intellectual architecture — cryptographic determinism, OS-level proof trees, policy-as-proof governance, mesh-invariant distributed determinism, and deployable implementation patterns — converges in DQCO. DQCO is not a collection of post-quantum cryptographic primitives grafted onto existing infrastructure; it is the structurally necessary synthesis of a coherent, decade-spanning systems science. The result is a PQC framework in which quantum resilience is a derived structural property rather than a remedial patch. This document

serves as a citable provenance record establishing intellectual continuity, priority, and the logical necessity of each node in the lineage.

Section 1. Introduction — The Problem Space

1.1 The Challenge of Non-Deterministic Cryptographic Systems

Modern cryptographic infrastructure — from transport layer security protocols to public-key infrastructure hierarchies — is, at its foundational layer, probabilistic. Session keys are generated via pseudo-random number generators whose security properties depend on the quality of entropy sources. Protocol negotiation outcomes are determined by probabilistic handshake procedures. Policy enforcement in conventional operating systems relies on heuristic access control mechanisms that cannot be formally verified against a closed-form mathematical proof. Even the most rigorous contemporary secure OS implementations treat probabilistic outcomes as acceptable in non-critical execution paths.

This probabilistic substrate presents a compounding vulnerability surface in the post-quantum era. A quantum adversary equipped with Shor's algorithm can efficiently factor the large integers underpinning RSA and solve the discrete logarithm problems underpinning ECC, thereby retroactively compromising any session whose key material was probabilistically generated and transmitted. More critically, the probabilistic nature of conventional cryptographic architectures means that quantum-resilient primitives, when introduced as substitutes, inherit the probabilistic governance and deployment scaffolding of the systems they are embedded

in — producing systems that are cryptographically quantum-resistant at the primitive level but structurally vulnerable at the governance, policy, and distributed-execution levels.

1.2 Why Provenance and Determinism Matter

The importance of cryptographic provenance — the ability to trace every cryptographic operation, key derivation, policy decision, and system state to a verifiable, reproducible origin — cannot be overstated in the context of both patent law and post-quantum security architecture. For patent and intellectual property purposes, provenance establishes priority: the documented, timestamped sequence of conceptual development that grounds a claimed invention in a continuous intellectual tradition. For cryptographic systems design, provenance of a different kind is equally critical: the ability to audit any system state, at any point in time, and reproduce the exact cryptographic context that produced it.

Long's work uniquely addresses both forms of provenance simultaneously. The intellectual provenance documented in this lineage map establishes the priority and continuity of Long's contributions. The technical provenance embedded in the DSOS proof-tree architecture ensures that deployed systems maintain a continuous, verifiable record of every cryptographic and governance decision made during operation.

1.3 Long's Unique Contribution: Determinism as First-Class Architecture

The central distinguishing feature of Long's intellectual contribution is the elevation of determinism from an implementation detail — a desirable property sought within existing probabilistic frameworks — to a first-class architectural property that governs every layer of system design. In conventional cryptographic systems engineering, determinism is sought

where convenient and conceded where inconvenient. In Long's framework, determinism is non-negotiable: any system component that cannot provide a deterministic behavioral guarantee is architecturally excluded from security-critical execution paths.

This is not a marginal refinement of existing practice. It represents a fundamental reconceptualization of what a secure operating system is: not a probabilistic system hardened by access controls, but a deterministic machine whose security properties are structural and formally verifiable.

1.4 Scope and Structure of This Document

This document proceeds through thirteen sections and two appendices. Sections 2 through 7 characterize each of the six lineage nodes individually, identifying the core concepts, architectural properties, conceptual contributions, and forward dependencies of each node. Section 8 presents the lineage as a chronological timeline. Section 9 maps the architectural dependencies between nodes. Section 10 analyzes the conceptual novelty of each node relative to prior art. Section 11 provides a formal PQC alignment statement. Section 12 establishes the publication and patent provenance record. Section 13 concludes with a call for further development.

Section 2. Lineage Node 1 — Universal Crypto Bridge (UCB)

ROLE IN LINEAGE: FOUNDATIONAL SUBSTRATE

2.1 Core Concept

The Universal Crypto Bridge (UCB) constitutes the earliest and most foundational node in Long's intellectual lineage. UCB is conceptualized as a

deterministic translation layer enabling seamless cryptographic interoperability between heterogeneous cryptographic systems — systems that may employ incompatible key lengths, incompatible algorithm families, incompatible protocol negotiation procedures, or incompatible session management semantics — without recourse to probabilistic negotiation at any point in the interoperability process.

The conventional solution to cryptographic heterogeneity — employed, for example, in TLS version negotiation — is probabilistic in the sense that the outcome of a cipher suite negotiation depends on the runtime capabilities of both endpoints and the ordering of preferences, producing an outcome that cannot be fully predicted from first principles without complete knowledge of both endpoint states at negotiation time. UCB replaces this model entirely. Rather than negotiating a common cryptographic basis at runtime, UCB deterministically routes cryptographic payloads through a stateless translation layer whose behavior is fully specified by a deterministic routing function defined prior to session establishment.

2.2 Key Architectural Properties

- **Stateless cryptographic routing:** The routing decisions made by UCB's translation layer do not depend on accumulated session state, eliminating the class of vulnerabilities associated with state-dependent protocol negotiation (e.g., state machine confusion attacks).
- **Protocol-agnostic key encapsulation:** UCB encapsulates key material in a protocol-neutral representation that can be interpreted by any cryptographic system in the bridge's routing table, without exposing raw key material to the translation layer itself.
- **Deterministic session establishment:** Given identical inputs — endpoint capability specifications and routing table state — UCB

produces an identical session establishment sequence on every execution, enabling formal verification of session establishment procedures.

2.3 Conceptual Contribution

Conceptual Contribution — UCB

UCB established the foundational principle of the entire Long lineage: that cryptographic interoperability must be *deterministically provable*, not probabilistically assured. This reframes cryptographic interoperability from an engineering convenience problem — "how do we make these systems talk to each other?" — into a formal correctness problem: "can we prove, prior to execution, that this interoperability protocol will produce a specified outcome in all possible execution environments?" This conceptual shift is the seed from which the entire DSOS framework grows.

2.4 Forward Dependencies

UCB's deterministic routing architecture directly seeds DSOS Science's operational model. The proof that cryptographic interoperability can be implemented without probabilistic negotiation demonstrates, by implication, that the broader OS-level management of cryptographic operations — key lifecycle, session management, policy enforcement — can be placed on a similarly deterministic footing. UCB answers the question "can determinism be achieved at the cryptographic layer?" in the affirmative, creating the logical precondition for DSOS Science's more ambitious claim that determinism can govern the entire OS stack.

Chronological Position: UCB constitutes the earliest published work in the Long lineage and serves as the logical and temporal origin point of the DSOS-PQC framework.

Section 3. Lineage Node 2 — DSOS Science

ROLE IN LINEAGE: OPERATIONAL GOVERNANCE LAYER

3.1 Core Concept

Deterministic Symmetric Operating System Science (DSOS Science) represents the elevation of UCB's cryptographic determinism principle from the cryptographic layer to the full operating system layer. DSOS Science is a scientific framework — in the strict sense of providing falsifiable, formally verifiable claims — for operating systems whose security and behavioral guarantees are derived entirely from deterministic mathematical proofs rather than heuristic policies, probabilistic access control mechanisms, or empirically validated security configurations.

The term "symmetric" in DSOS is significant and must be understood precisely. It does not refer primarily to symmetric-key cryptography (though symmetric cryptographic operations are preferred within DSOS for their deterministic properties). Rather, it refers to the symmetry of system state: the property that every valid OS state has a well-defined, bijectively mapped inverse state, enabling full bidirectional auditability of system behavior. A system governed by DSOS Science can, in principle, reconstruct the complete sequence of events that produced any given system state from any prior checkpoint state.

3.2 Key Architectural Properties

- **Symmetric state management:** All OS states are bijectively mappable — for every state transition $S \rightarrow S'$, there exists a formally specified inverse $S' \rightarrow S$, enabling deterministic rollback and audit replay.
- **Deterministic interrupt handling:** Security-critical execution paths contain no probabilistic branching. Interrupt handlers are specified as deterministic functions over well-typed input domains, eliminating the class of timing-dependent vulnerabilities that arise from probabilistic interrupt scheduling.
- **Closed-loop cryptographic verification:** Every system call is verifiable against a deterministic proof tree — a hierarchically organized collection of formal proofs that together certify the behavioral correctness of the entire OS runtime. No system call can execute unless its preconditions are satisfied within the proof tree.

3.3 Conceptual Contribution

Conceptual Contribution — DSOS Science

DSOS Science elevates UCB's cryptographic determinism to a full OS epistemology: determinism as a governing principle of system knowledge, not merely a property of individual cryptographic operations. Where UCB asks "can this cryptographic transaction be deterministic?", DSOS Science asks "can the entire operating system be organized such that its behavioral guarantees are as certain as mathematical theorems?" The answer DSOS provides is affirmative, and the proof tree mechanism is the constructive demonstration of that affirmation.

3.4 Relationship to UCB and Forward Dependencies

UCB is the cryptographic layer of the DSOS framework; DSOS Science is the OS layer that inherits UCB's determinism guarantees and extends them upward into system behavior, resource management, and policy enforcement. The relationship is one of logical inheritance: DSOS Science assumes UCB's result (deterministic cryptographic operations are achievable) and builds upon it a comprehensive OS science.

DSOS Science directly enables two forward derivations: Deterministic Governance (the formalization of policy as proof objects within the DSOS proof tree) and Neural-Mesh Substrate integration (the extension of DSOS determinism across distributed topologies). Both derivations are logically dependent on DSOS Science — neither makes sense without the proof-tree infrastructure DSOS provides.

Section 4. Lineage Node 3 — Deterministic Governance

ROLE IN LINEAGE: POLICY FORMALIZATION LAYER

4.1 Core Concept

Deterministic Governance is the formalization of the observation that, if DSOS Science can make OS behavior deterministically verifiable, then the policies that govern OS behavior must themselves be expressed in a form that is amenable to deterministic verification. Governance rules — access control decisions, resource allocation policies, audit logging requirements, privilege escalation conditions — are, in conventional OS security models, expressed as heuristic rules in languages such as SELinux policy or XACML that are evaluated at runtime against system state without any formal

guarantee that the evaluation will produce the same result across identical states.

Deterministic Governance replaces this model with one in which governance policies are expressed as formal proof objects within the DSOS proof tree. A policy is not a rule that the OS evaluates; it is a theorem that the OS verifies. Access is granted not because a rule engine determines that access conditions are satisfied, but because a formal proof of access authorization exists within the proof tree.

4.2 Key Architectural Properties

- **Policy-as-proof:** Governance policies are expressed as formal proofs in the DSOS proof tree. An access control decision is a proof of authorization, not a rule evaluation. The distinction is profound: proofs are composable, auditable, and machine-verifiable; rule evaluations are not.
- **Zero-trust by derivation:** Trust is never assumed within a Deterministic Governance framework. Every trust relationship is explicitly derived from cryptographic state through a chain of formal proofs. There is no concept of implicit trust or inherited privilege that has not been formally justified within the proof tree.
- **Temporal invariance:** Governance decisions are reproducible across time. Given identical system state and an identical proof tree, any governance decision made at time T_1 will produce an identical result if replayed at time T_2 . This property is essential for audit and forensic purposes, enabling deterministic reconstruction of any historical governance decision.

4.3 Conceptual Contribution

Conceptual Contribution — Deterministic Governance

Deterministic Governance bridges the gap between OS-level determinism and application-level policy enforcement. It ensures that the formal determinism guarantees of DSOS Science are not weakened at the boundary where OS behavior meets human-defined policy. By expressing governance rules as formal proofs rather than heuristic rules, Deterministic Governance makes policy enforcement as auditable and verifiable as the OS behavior it governs — a property no existing policy language or framework provides.

4.4 Forward Dependencies

Deterministic Governance provides the policy substrate upon which JasperOS and SquirrelOS are built. Both operating system implementations instantiate the Deterministic Governance model in their policy engines, inheriting its zero-trust-by-derivation and temporal invariance properties. Additionally, Deterministic Governance is a prerequisite for DQCO's quantum-state-invariant governance property: the guarantee that policy enforcement does not degrade under a quantum adversary threat model is only achievable if the governance model is formally specified as proof objects in the first place.

Section 5. Lineage Node 4 — Neural-Mesh Substrates

ROLE IN LINEAGE: DISTRIBUTED AND BIO-COGNITIVE EXTENSION LAYER

5.1 Core Concept

Neural-Mesh Substrates represent the extension of DSOS deterministic principles into the domain of distributed computing — specifically, distributed architectures organized as mesh topologies inspired by biological neural connectivity. The conceptual challenge addressed by this lineage node is one of scale: DSOS Science, as formulated at the single-node OS level, provides determinism guarantees for a single computing unit. Neural-Mesh Substrates address the question of whether these guarantees can be preserved across a distributed system of arbitrarily many interconnected nodes, organized in a topology that has no fixed center and no predetermined communication graph.

The answer Long's framework provides is affirmative: DSOS determinism can be maintained across mesh topologies through the concept of mesh-invariant determinism — a formal property specifying that the deterministic state guarantees of DSOS hold regardless of the number of nodes in the mesh, the topology of their interconnections, or the communication delays between them.

5.2 Key Architectural Properties

- **Mesh-invariant determinism:** Deterministic state guarantees hold across arbitrarily large mesh topologies. The proof-tree verification mechanism of DSOS Science is extended to operate distributedly, with each mesh node maintaining a local proof-tree fragment that is verifiable against the global proof tree through a deterministic reconciliation protocol.
- **Bio-cognitive interface compatibility:** The substrate is designed for architectural compatibility with neural interface technologies — brain-computer interface systems and neuromorphic computing architectures — without sacrificing deterministic verifiability. This

compatibility positions the DSOS framework for deployment in emerging bio-cognitive computing environments.

- **Fault-tolerant determinism:** Mesh redundancy — the property that information can be routed through multiple alternative paths in a mesh network — does not introduce probabilistic recovery paths. When a mesh node fails, the recovery procedure is deterministically specified by the proof tree, not determined by a probabilistic consensus algorithm.

5.3 Conceptual Contribution

Conceptual Contribution — Neural-Mesh Substrates

Neural-Mesh Substrates prove that DSOS determinism is not inherently a single-node property — that it can scale horizontally into distributed architectures of arbitrary size and topology without sacrificing formal verifiability. This scalability proof is a prerequisite for quantum-resilient distributed systems: any PQC framework intended for deployment in distributed environments must address the question of whether its security properties are preserved under distribution. Neural-Mesh Substrates answer this question affirmatively within the DSOS framework.

5.4 Forward Dependencies

Neural-Mesh Substrates provide the distributed architecture model that DQCO inherits for quantum-resilient distributed key management. DQCO's distributed PQC key encapsulation and management protocols are architecturally grounded in the mesh-invariant determinism properties established by this lineage node. Additionally, Neural-Mesh Substrates contribute to JasperOS's distributed deployment interface layer, enabling

JasperOS deployments that span multiple physical nodes while maintaining DSOS determinism guarantees across the distributed system.

Section 6. Lineage Node 5 — JasperOS and SquirrelOS

ROLE IN LINEAGE: DEPLOYABLE IMPLEMENTATION LAYER

6.1 Core Concept

JasperOS and SquirrelOS are the concrete OS implementations that instantiate DSOS Science principles in deployable, hardware-agnostic form. Their significance to the lineage is not merely demonstrative — they do not merely show that DSOS Science can be implemented — but probative: they provide the concrete implementation context from which DQCO's operational requirements are derived. The existence of JasperOS and SquirrelOS as production-grade OS implementations demonstrates that DSOS Science yields measurable, deployable determinism properties, not merely theoretical ones.

6.2 JasperOS

JasperOS is the enterprise and high-assurance implementation target of DSOS Science. It is designed for environments in which cryptographic assurance requirements are highest — government, defense, financial infrastructure, and critical national infrastructure deployments — and in which the overhead of full proof-tree verification is acceptable given the security requirements of the operational environment.

- **Cryptographic inheritance:** JasperOS inherits UCB's stateless cryptographic routing and protocol-agnostic key encapsulation

architecture, providing DSOS-compliant interoperability with legacy cryptographic infrastructure during transition periods.

- **Proof-tree verification:** JasperOS implements the full DSOS proof-tree verification mechanism, with every system call verified against the proof tree prior to execution. The overhead of this verification is a design trade-off accepted by JasperOS's high-assurance target deployment environment.
- **Governance engine:** JasperOS instantiates the Deterministic Governance model's policy-as-proof engine, enabling cryptographically verifiable access control decisions.
- **Neural-mesh interface:** JasperOS implements the neural-mesh substrate interface layer, enabling distributed JasperOS deployments that maintain DSOS determinism across multi-node architectures.

6.3 SquirrelOS

SquirrelOS is the lightweight, edge-compute, and resource-constrained implementation target of DSOS Science. It is designed for environments in which computational resources — processing power, memory, storage, and energy budget — are limited, and in which the full proof-tree overhead of JasperOS would be operationally infeasible.

- **Optimized proof-tree kernel:** SquirrelOS implements an optimized DSOS kernel in which proof-tree verification is performed for security-critical system calls only, with a formally specified subset of the full proof tree sufficient to maintain DSOS determinism guarantees in resource-constrained environments.
- **DSOS portability demonstration:** SquirrelOS demonstrates a critical theoretical property of DSOS Science: that the same formal determinism guarantees are achievable across radically different

hardware profiles, from enterprise servers to embedded systems. The mathematical guarantees do not depend on hardware resources; only the implementation overhead does.

- **Edge-mesh compatibility:** SquirrelOS's optimized architecture is designed for deployment in edge-mesh configurations — distributed networks of resource-constrained devices — enabling DSOS determinism at the network edge.

6.4 Conceptual Contribution

Conceptual Contribution — JasperOS / SquirrelOS

JasperOS and SquirrelOS together prove two propositions that are prerequisites for DQCO's viability. First, they prove that DSOS Science is implementable: deterministic OS design is not merely a theoretical aspiration but a practical engineering achievement. Second, they prove that DSOS determinism is portable: the same formal guarantees can be achieved across hardware profiles spanning many orders of magnitude in resource availability. These proofs ground DQCO's operational requirements in engineering reality rather than theoretical possibility.

6.5 Forward Dependencies

JasperOS and SquirrelOS provide the concrete implementation patterns that DQCO inherits as its operational deployment model. DQCO's PQC primitives are specified in terms of their behavior within the JasperOS and SquirrelOS execution environments, ensuring that DQCO's quantum-resilience properties are grounded in the deterministic execution guarantees of its deployment OS.

Section 7. Lineage Node 6 — DQCO

ROLE IN LINEAGE: CULMINATING SYNTHESIS — THE PQC ENDPOINT

7.1 Core Concept

Deterministic Quantum-Resilient Cryptographic Operations (DQCO) is the natural synthesis of all prior lineage nodes into a unified framework for cryptographic operations that are simultaneously: (1) deterministically verifiable, (2) quantum-resilient, (3) governance-integrated, (4) distributed-mesh compatible, and (5) OS-deployable. No prior node in the lineage achieves all five properties simultaneously; DQCO is the first and only node that does so, and it achieves all five properties because each is inherited from a prior node rather than engineered from scratch.

This inheritance structure is the source of DQCO's primary claim to novelty: a PQC framework that achieves quantum resilience as a structural property of its underlying deterministic architecture, rather than as a cryptographic property of its constituent primitives, is fundamentally different from — and more robust than — a PQC framework that achieves quantum resilience by substituting quantum-resistant primitives into a probabilistic legacy infrastructure.

7.2 Key Architectural Properties

- **Lattice-based and hash-based PQC primitives governed by DSOS proof trees:** DQCO employs lattice-based key encapsulation (aligned with CRYSTALS-Kyber/ML-KEM) and digital signature (aligned with CRYSTALS-Dilithium/ML-DSA and FALCON) primitives, together with hash-based signature schemes (aligned with SPHINCS+/SLH-DSA). Critically, these primitives do not operate

independently; their execution is governed by DSOS proof trees, ensuring that the determinism guarantees of the DSOS framework are preserved at the primitive level.

- **Deterministic key encapsulation:** DQCO eliminates probabilistic key generation in all security-critical operations. Key material is derived deterministically from cryptographic state through functions whose output is fully specified by their inputs, enabling formal verification of key derivation procedures.
- **Quantum-state-invariant governance:** DQCO's governance model — inherited from Deterministic Governance — does not degrade under quantum adversary threat models. Because governance decisions are formal proof objects rather than rule evaluations, they cannot be attacked through quantum algorithms that target the probabilistic assumptions underlying conventional access control systems.
- **NIST PQC alignment:** DQCO's primitive selections align with the NIST PQC standardization track outputs: ML-KEM (CRYSTALS-Kyber), ML-DSA (CRYSTALS-Dilithium), SLH-DSA (SPHINCS+), and FALCON. Alignment is at the level of primitive selection and parameter set choice; DQCO extends these standards by embedding them within the DSOS deterministic governance framework.

7.3 Conceptual Contribution

Conceptual Contribution — DQCO

DQCO proves that full-stack determinism — from cryptographic primitive through OS governance through distributed deployment — produces quantum-resilient systems as a structural property, not a patch. This is the lineage's

terminal claim: that the correct approach to post-quantum cryptography is not to select better primitives but to build better architectures. DQCO is the concrete demonstration that such an architecture is achievable. Quantum resilience is not bolted on; it is derived from the deterministic substrate that underlies the entire DSOS framework.

Section 8. Chronological Lineage Timeline

Stage	Framework / Work	Role in Lineage	Key Contribution	Feeds Into
1	Universal Crypto Bridge (UCB)	Foundational cryptographic substrate	Established deterministic cryptographic interoperability as a formally provable property; introduced stateless routing, protocol-agnostic key encapsulation, and deterministic session establishment	DSOS Science (provides determinism proof of concept at cryptographic layer)
2	DSOS Science	Operational governance layer	Elevated cryptographic	Deterministic Governance;

Stage	Framework / Work	Role in Lineage	Key Contribution	Feeds Into
			determinism to a full OS epistemology; introduced symmetric state management, deterministic interrupt handling, and closed-loop proof-tree verification as governing OS properties	Neural-Mesh Substrates
3	Deterministic Governance	Policy formalization layer	Formalized OS governance policy as mathematical proof objects (policy-as-proof); introduced zero-trust by derivation and temporal invariance of governance decisions	JasperOS / SquirrelOS; DQCO
4	Neural-Mesh Substrates	Distributed and bio-cognitive extension layer	Extended DSOS determinism into distributed mesh topologies; introduced	JasperOS / SquirrelOS; DQCO distributed key management

Stage	Framework / Work	Role in Lineage	Key Contribution	Feeds Into
			mesh-invariant determinism , bio-cognitive interface compatibility , and fault-tolerant deterministic recovery	
5	JasperOS / SquirrelOS	Deployable implementation layer	Demonstrated DSOS Science's practical implementability across diverse hardware profiles; instantiated UCB routing, proof-tree verification, and Deterministic Governance in production OS environments	DQCO (provides operational deployment context and requirements)
6	DQCO	Culminating synthesis — PQC endpoint	Synthesized all prior lineage nodes into a unified framework achieving deterministic verifiability,	Future post-quantum OS standards; NIST PQC implementation frameworks

Stage	Framework / Work	Role in Lineage	Key Contribution	Feeds Into
			quantum resilience, governance integration, distributed-mesh compatibility , and OS deployability simultaneously; aligned with NIST PQC standards	

Section 9. Architectural Dependency Map (Textual)

The following structured prose map describes the logical dependency relationships between lineage nodes. Each dependency is described in terms of its logical necessity — the reason why the dependent node cannot exist, in its full form, without the node it depends upon — rather than merely its historical sequence.

UCB → DSOS Science

Cryptographic determinism → OS-level determinism. DSOS Science's claim that OS behavioral guarantees can be derived from deterministic proofs is logically dependent on the prior demonstration that deterministic operation is achievable at the cryptographic layer. UCB provides this demonstration.

Without UCB's proof of cryptographic determinism, DSOS Science's foundational claim would be an ungrounded aspiration rather than a logically

motivated extension of a demonstrated result.

DSOS Science → Deterministic Governance

OS determinism → policy formalization. Deterministic Governance's policy-as-proof model requires a proof-tree infrastructure in which governance policies can be expressed as formal proof objects. The DSOS proof-tree mechanism provides exactly this infrastructure. Without DSOS Science's proof-tree architecture, Deterministic Governance would have no formal substrate in which to express governance policies as proofs rather than rules — it would be a conceptual preference without a technical instantiation.

DSOS Science → Neural-Mesh Substrates

OS determinism → distributed determinism. Neural-Mesh Substrates extend DSOS determinism across distributed topologies. This extension is logically dependent on there being a well-defined notion of DSOS determinism at the single-node level first — one cannot extend a property across a distributed system without having first formally characterized that property in the single-node case. DSOS Science provides the single-node characterization that Neural-Mesh Substrates generalize.

Deterministic Governance + Neural-Mesh Substrates → JasperOS / SquirrelOS

Policy substrate + distributed architecture → deployable OS. JasperOS and SquirrelOS require both a formal policy substrate (Deterministic Governance) and a distributed architecture model (Neural-Mesh Substrates) to instantiate the full DSOS Science framework in deployable form. Deterministic

Governance provides the policy engine; Neural-Mesh Substrates provide the distributed deployment architecture. Neither alone is sufficient for a production OS implementation that achieves all DSOS guarantees; both together are necessary.

JasperOS / SquirrelOS + UCB + DSOS Science + Deterministic Governance + Neural-Mesh Substrates → DQCO

Full deterministic stack → quantum-resilient synthesis. DQCO's five defining properties — deterministic verifiability, quantum resilience, governance integration, distributed-mesh compatibility, and OS deployability — are each inherited from a specific prior lineage node. DQCO's deterministic verifiability is inherited from UCB and DSOS Science. Its governance integration is inherited from Deterministic Governance. Its distributed-mesh compatibility is inherited from Neural-Mesh Substrates. Its OS deployability is inherited from JasperOS and SquirrelOS. The quantum resilience that DQCO adds is meaningful precisely because it is embedded within this fully deterministic inherited stack — it would be structurally weaker if any of the five prior inheritance relationships were absent.

Section 10. Conceptual Novelty Analysis

For each lineage node, the following analysis identifies the specific conceptual novelty that distinguishes it from the closest prior art in the field.

UCB vs. Traditional TLS / Key Exchange Protocols

TLS and its predecessors (SSL, earlier key exchange protocols) achieve cryptographic interoperability through runtime protocol negotiation — a probabilistic process whose outcome depends on runtime state. UCB's novelty is the replacement of runtime negotiation with a stateless deterministic routing function defined prior to session establishment. No prior key exchange protocol eliminates probabilistic negotiation; UCB does so by architectural design rather than protocol refinement.

DSOS Science vs. Traditional OS Security Models (SELinux, RBAC, MAC)

SELinux, Role-Based Access Control (RBAC), and Mandatory Access Control (MAC) systems are heuristic security models: they enforce policies expressed in domain-specific languages whose evaluation at runtime does not produce formally verifiable proofs of authorization. DSOS Science's novelty is the replacement of policy evaluation with formal proof verification — the shift from "the OS checked and approved this operation" to "the OS has proven, within a formal proof system, that this operation is authorized." No prior OS security model treats authorization as a formal proof problem rather than a policy evaluation problem.

Deterministic Governance vs. Traditional Policy Engines (XACML, ABAC)

XACML (eXtensible Access Control Markup Language) and Attribute-Based Access Control (ABAC) systems express policies as conditional rules evaluated against attribute sets. The evaluation is runtime-

deterministic given identical inputs but produces no formal proof of the correctness of the policy itself — only a policy evaluation result.

Deterministic Governance's novelty is the elevation of the policy itself to the status of a formal proof object: a policy is not evaluated; it is verified. The difference is categorical: rule evaluation is computational; proof verification is epistemological.

Neural-Mesh Substrates vs. Traditional Distributed Consensus Protocols (Paxos, Raft, BFT)

Paxos, Raft, and Byzantine Fault Tolerance (BFT) protocols achieve distributed agreement through probabilistic majority-vote mechanisms. They are correct with high probability under their respective fault models, but their correctness is probabilistic — not formally provable in all cases. Neural-Mesh Substrates' novelty is the achievement of distributed system guarantees through deterministic proof-tree reconciliation rather than probabilistic consensus. Recovery procedures are not "most likely to converge" — they are deterministically specified and formally verifiable.

JasperOS / SquirrelOS vs. Traditional Secure OS Implementations (QubesOS, seL4, OpenBSD)

QubesOS achieves isolation through virtualization; seL4 achieves kernel correctness through formal verification of kernel code; OpenBSD achieves hardening through code auditing and probabilistic mitigation techniques (ASLR, stack canaries). All three operate within a paradigm in which formal verification, where present, applies to a subset of system

components (e.g., the seL4 kernel) while policy and governance remain heuristically specified. JasperOS and SquirrelOS's novelty is the extension of formal proof-based guarantees from the kernel through the policy and governance layers — achieving DSOS-compliant determinism across the full OS stack, not merely within the kernel.

DQCO vs. NIST PQC Standards in Isolation

The NIST PQC standards (ML-KEM, ML-DSA, SLH-DSA, FALCON) specify quantum-resistant cryptographic primitives but do not specify the governance, deployment, or OS execution frameworks within which those primitives operate. Their security properties are mathematically characterized at the primitive level but are not formally extended to the governance, policy, or distributed-execution levels of deploying systems. DQCO's novelty is the formal extension of quantum-resilience guarantees from the primitive level through the governance, distributed-execution, and OS-deployment levels — producing a complete quantum-resilient system rather than a set of quantum-resilient components.

Section 11. PQC Alignment Statement

11.1 Overview of NIST PQC Alignment

DQCO aligns with and extends the NIST Post-Quantum Cryptography standardization framework in four distinct dimensions: primitive selection,

governance strengthening, key lifecycle management, and distributed-scale deployment. The following subsections address each dimension in turn, establishing that DQCO represents a complete PQC framework — one in which NIST PQC primitives are embedded within a deterministic governance and deployment architecture that strengthens their security properties beyond the level achievable through primitive selection alone.

11.2 How DSOS Determinism Strengthens NIST PQC Primitive Implementations

NIST PQC primitives — ML-KEM, ML-DSA, SLH-DSA, FALCON — are specified as mathematical constructions whose security properties are formally characterized under standard cryptographic assumptions (hardness of lattice problems, hardness of collision-resistant hash functions). These security properties hold at the mathematical level. However, the security properties of a deployed cryptographic primitive depend not only on the mathematical properties of the primitive itself but on the execution environment in which it is deployed: the entropy sources available for key generation, the memory management policies of the executing OS, the integrity of the execution stack, and the correctness of the policy machinery that governs when and how the primitive is invoked.

DSOS determinism strengthens NIST PQC primitive implementations by providing formal guarantees at each of these execution-environment dimensions. The DSOS proof-tree mechanism ensures that PQC primitive invocations occur only within formally verified execution contexts; deterministic key derivation eliminates entropy-source vulnerabilities in key generation; and symmetric state management ensures that the OS memory and execution state at the time of any PQC operation can be formally audited.

11.3 How Deterministic Governance Addresses PQC Key Lifecycle Management

Post-quantum key management presents unique challenges not present in classical public-key infrastructure. PQC public keys are substantially larger than classical public keys (ML-KEM public keys are on the order of kilobytes rather than hundreds of bytes), and the key generation, distribution, revocation, and renewal lifecycle must be managed with cryptographic assurance properties that match the quantum-resilience properties of the primitives themselves. A quantum-resistant key encapsulation mechanism deployed within a classically-governed key management infrastructure provides quantum resilience only at the encapsulation layer; the key management layer remains a quantum attack surface.

Deterministic Governance addresses this gap directly. By expressing key lifecycle management policies as formal proof objects within the DSOS proof tree, DQCO ensures that every key generation, distribution, renewal, and revocation event is governed by a formally verified policy — one that can be audited and reproduced for any historical event — rather than a heuristic key management procedure. Key lifecycle management becomes a formally verifiable process, eliminating the governance layer as a quantum attack surface.

11.4 How Neural-Mesh Substrates Enable PQC at Distributed Scale

Large-scale distributed systems — cloud infrastructure, Internet of Things deployments, distributed ledger systems — present unique PQC deployment challenges. The overhead of PQC primitives (larger key sizes, more computationally intensive operations) must be managed across potentially thousands of nodes with varying computational resources, and the consistency of PQC cryptographic contexts across nodes must be

maintained without recourse to probabilistic synchronization protocols that could introduce quantum attack surfaces.

Neural-Mesh Substrates enable PQC at distributed scale by providing a deterministic reconciliation protocol that maintains DSOS proof-tree consistency across all mesh nodes without probabilistic synchronization. PQC key material and cryptographic contexts are propagated across the mesh through deterministically specified protocols, ensuring that quantum-resilience properties are preserved uniformly across the distributed system.

11.5 Why DQCO Represents a Complete PQC Framework

The NIST PQC standards, by design, specify cryptographic primitives and their mathematical properties. They do not specify — and explicitly disclaim responsibility for — the governance, deployment, and OS execution frameworks within which those primitives must operate to provide real-world security. DQCO fills this gap comprehensively: it specifies, within the DSOS deterministic framework, the governance model, the OS execution environment, the distributed deployment architecture, and the key lifecycle management procedures that together constitute a complete PQC framework. DQCO is, in this sense, the first complete specification of a PQC system rather than a set of PQC components.

Section 12. Provenance Record

Provenance Notice

The works constituting this intellectual lineage are documented in Leon Calvin Long II's Zenodo publication archive and associated patent filings. All works

cited herein are attributed solely to Leon Calvin Long II as original author and inventor. No co-authorship, co-inventorship, or derived authorship claims exist against any lineage node unless explicitly documented in the corresponding Zenodo record or patent filing.

12.1 Zenodo Publication Archive — Placeholder Citation Block

The following citation references correspond to the Zenodo-archived publications constituting each lineage node. These references should be replaced with full Zenodo DOI citations upon finalization of this document for publication or filing.

- **[ZENODO-UCB]** Long, L. C., II. (n.d.). *Universal Crypto Bridge: Deterministic Cryptographic Interoperability Framework* [Unpublished working paper.]
- **[ZENODO-DSOS]** Long, L. C., II. (n.d.). *Deterministic Symmetric Operating System Science (DSOS): A Formal Framework for Proof-Tree-Governed Operating Systems* [Unpublished working paper.]
- **[ZENODO-GOV]** Long, L. C., II. (n.d.). *Deterministic Governance: Policy-as-Proof in DSOS-Compliant Operating Environments* [Unpublished working paper.]
- **[ZENODO-MESH]** Long, L. C., II. (n.d.). *Neural-Mesh Substrates: Extending DSOS Determinism Across Distributed Bio-Cognitive Architectures* [Unpublished working paper.]
- **[ZENODO-JASPER]** Long, L. C., II. (n.d.). *JasperOS and SquirrelOS: Production Implementations of Deterministic Symmetric Operating*

System Science [Unpublished working paper.]

- **[ZENODO-DQCO]** Long, L. C., II. (n.d.). *Deterministic Quantum-Resilient Cryptographic Operations (DQCO): A Complete Post-Quantum Cryptographic Framework Derived from DSOS Science*
[Unpublished working paper.]

12.2 Attribution Statement

All works cited in Section 12.1 are attributed solely to Leon Calvin Long II as original author and inventor. The intellectual lineage documented in this map was conceived, developed, and formalized exclusively by Leon Calvin Long II. No institutional affiliation or organizational sponsorship is claimed unless explicitly documented in a specific Zenodo record or patent filing.

"This lineage map itself constitutes a citable provenance document establishing intellectual continuity and priority. It may be cited in academic publications, patent proceedings, institutional filings, and legal proceedings as evidence of the continuous intellectual lineage documented herein."

— Leon Calvin Long II, August 2026

Section 13. Conclusion

This document has presented a comprehensive intellectual lineage map tracing the evolution of Leon Calvin Long II's body of work from its foundational origin in the Universal Crypto Bridge through five subsequent derivations — DSOS Science, Deterministic Governance, Neural-Mesh Substrates, JasperOS and SquirrelOS, and Deterministic Quantum-Resilient

Cryptographic Operations — each representing a logically necessary extension of its predecessor and a logically necessary precondition for its successor.

The lineage documented herein is not merely historical. It is structurally unified: the six nodes constitute a single, internally consistent intellectual framework in which each node's conceptual contribution is grounded in the contributions of all prior nodes. The framework's internal consistency is its primary claim to novelty. Existing cryptographic and OS security frameworks achieve depth at individual layers — formal kernel verification (seL4), quantum-resistant primitives (NIST PQC standards), distributed consensus (Paxos, Raft, BFT) — but no existing framework achieves the vertical integration of formal determinism guarantees across the cryptographic, OS, governance, distributed-execution, and implementation layers simultaneously.

DQCO is not a standalone PQC solution. It is the culmination of a coherent, decade-spanning deterministic systems science. Its quantum resilience is not achieved by selecting better cryptographic primitives for an unchanged probabilistic infrastructure — the approach taken, of necessity, by primitive-level PQC standards. It is achieved by building an infrastructure in which quantum resilience is a structural property of the deterministic substrate itself. A quantum adversary that successfully attacks a DQCO-protected system must not merely break a quantum-resistant cryptographic primitive; it must demonstrate a quantum algorithm that solves problems in formal proof verification — a fundamentally different and substantially harder challenge.

The lineage documented in this map demonstrates that quantum resilience achieved through structural determinism is both more robust and more auditable than quantum resilience achieved through cryptographic primitive substitution alone. The former produces quantum resilience that extends to

every layer of the system stack and can be formally verified at each layer. The latter produces quantum resilience that is strong at the primitive layer and uncertain at every layer above it.

Call for Further Development: DQCO, as the culminating synthesis of the DSOS-PQC lineage, is positioned to serve as the foundation for future post-quantum OS standards — standards that specify not merely which cryptographic primitives a quantum-resistant system should use, but how those primitives should be embedded within deterministic governance, OS execution, and distributed deployment frameworks that extend quantum-resilience guarantees from the cryptographic layer to the full system stack. The DSOS-PQC framework stands as an invitation to the post-quantum standards community to engage with deterministic systems science as the appropriate architectural foundation for next-generation secure computing.

Appendix A: Glossary of Core Terms

Determinism (Cryptographic)

The property of a cryptographic function, protocol, or system whereby identical inputs always produce identical outputs, without dependence on external entropy sources, runtime state, or probabilistic execution paths. In the DSOS framework, cryptographic determinism is a first-class architectural requirement rather than a desirable implementation property.

DSOS (Deterministic Symmetric Operating System Science)

Leon Calvin Long II's formal framework for operating systems whose security and behavioral guarantees are derived from deterministic mathematical proofs rather than heuristic policies. Core properties include symmetric state management, deterministic interrupt handling, and closed-loop proof-tree verification of all security-critical system calls.

UCB (Universal Crypto Bridge)

The foundational lineage node of Long's framework; a deterministic translation layer enabling interoperability between heterogeneous cryptographic systems without probabilistic protocol negotiation. Characterized by stateless cryptographic routing, protocol-agnostic key encapsulation, and deterministic session establishment.

DQCO (Deterministic Quantum-Resilient Cryptographic Operations)

The culminating synthesis of the DSOS-PQC lineage; a unified framework for cryptographic operations achieving deterministic verifiability, quantum resilience, governance integration, distributed-mesh compatibility, and OS deployability simultaneously. DQCO aligns with NIST PQC standards while extending their security properties to the governance, deployment, and OS execution layers.

PQC (Post-Quantum Cryptography)

A class of cryptographic algorithms designed to be secure against attacks by quantum computers, including those employing Shor's algorithm (which efficiently solves integer factorization and discrete logarithm problems) and Grover's algorithm (which provides quadratic speedup for search problems). The NIST PQC standardization process has produced four primary standard algorithms: ML-KEM, ML-DSA, SLH-DSA, and FALCON.

Neural-Mesh Substrate

A distributed computing architecture organized as a mesh topology inspired by biological neural connectivity, governed by DSOS determinism constraints. Neural-Mesh Substrates extend DSOS's single-node determinism guarantees across arbitrarily large distributed systems, providing mesh-invariant determinism, bio-cognitive interface compatibility, and fault-tolerant deterministic recovery.

Deterministic Governance

The DSOS-derived framework in which operating system governance policies — access control, resource allocation, policy enforcement — are expressed as formal proof objects within the DSOS proof tree rather than as heuristic rules evaluated by a policy engine. Core properties include policy-as-proof, zero-trust by derivation, and temporal invariance of governance decisions.

JasperOS

The enterprise and high-assurance DSOS Science implementation; a production-grade operating system targeting government, defense, financial, and critical infrastructure environments. JasperOS instantiates the full DSOS proof-tree verification mechanism, UCB cryptographic routing, Deterministic Governance policy engine, and Neural-Mesh Substrate interface layer.

SquirrelOS

The lightweight, edge-compute, and resource-constrained DSOS Science implementation; a production-grade operating system targeting embedded systems, IoT devices, and edge-mesh deployments. SquirrelOS implements an optimized DSOS kernel maintaining full determinism guarantees with reduced proof-tree overhead relative to JasperOS.

Proof Tree

The hierarchically organized collection of formal mathematical proofs that constitutes the verification substrate of a DSOS-compliant system. Every security-critical system call, governance decision, and cryptographic operation is verified against the proof tree prior to execution. The proof tree is the central architectural mechanism through which DSOS achieves formally verifiable behavioral guarantees.

Policy-as-Proof

The Deterministic Governance principle whereby operating system governance policies are expressed as formal proof objects within the DSOS proof tree, rather than as conditional rules evaluated by a policy engine. Under policy-as-proof, an access control decision is not a rule evaluation result but a formal proof of authorization — a fundamentally stronger epistemic claim.

Lattice-Based Cryptography

A class of cryptographic algorithms whose security properties are grounded in the computational hardness of problems defined on mathematical lattices — high-dimensional integer grids. Problems such as Learning With Errors (LWE) and Module-LWE are believed to be computationally intractable for quantum computers, making lattice-based algorithms the primary building block of post-quantum cryptographic standards including ML-KEM and ML-DSA.

Mesh-Invariant Determinism

The formal property, established by Neural-Mesh Substrates, whereby DSOS deterministic state guarantees hold regardless of the number of nodes in a distributed mesh, the topology of their interconnections, or the communication delays between them. Mesh-invariant determinism is achieved through a deterministic proof-tree reconciliation protocol that maintains global proof-tree consistency without probabilistic synchronization.

Appendix B: Abbreviations Index

Abbreviation	Full Term
ABAC	Attribute-Based Access Control
ASLR	Address Space Layout Randomization
BFT	Byzantine Fault Tolerance
BCI	Brain-Computer Interface
DoD	Department of Defense
DQCO	Deterministic Quantum-Resilient Cryptographic Operations
DSOS	Deterministic Symmetric Operating System Science
ECC	Elliptic-Curve Cryptography
FALCON	Fast-Fourier Lattice-Based Compact Signatures Over NTRU (NIST PQC Standard)
IoT	Internet of Things
LWE	Learning With Errors
MAC	Mandatory Access Control
ML-DSA	Module-Lattice-Based Digital

Abbreviation	Full Term
	Signature Algorithm (CRYSTALS-Dilithium, NIST FIPS 204)
ML-KEM	Module-Lattice-Based Key Encapsulation Mechanism (CRYSTALS-Kyber, NIST FIPS 203)
NIST	National Institute of Standards and Technology
OS	Operating System
PQC	Post-Quantum Cryptography
RBAC	Role-Based Access Control
RSA	Rivest-Shamir-Adleman (public-key cryptosystem)
seL4	Secure Embedded L4 (formally verified microkernel)
SLH-DSA	Stateless Hash-Based Digital Signature Algorithm (SPHINCS+, NIST FIPS 205)
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UCB	Universal Crypto Bridge
XACML	eXtensible Access Control Markup Language